Building Production-Ready Agents

Welcome and Architecture Overview

Amazon Bedrock AgentCore

KavinSchool

Agenda

1. Course Objectives and Environment Setup
2. Introduction to Amazon Bedrock AgentCore
3. The Five Focus Capabilities
4. Enterprise Reference Architecture

1. Course Objectives & Setup

Course Objectives

- Bridge the gap between experimental AI scripts and production-grade systems.
- Understand the Amazon Bedrock AgentCore ecosystem.
- Build, secure, and operate agents using Runtime, Gateway, Identity, Memory, and Observability.

Introduce Yourself

In one chat message, type:

1. Your name
2. Your location (State) / timezone
3. Your title
4. Your tenure in IT and at your current company

> **Chat Waterfall**
>
> **Type your answer in the chat...**
>
> *Get ready and start typing: 3... 2... 1... BLAST*

Introduce Yourself: What We Learned

If your message included these four things, we have a clear picture of the room:

- **Your name**
- **Your location / timezone**, which state you're joining from
- **Your title**
- **Your tenure**, years in IT and years at your current company

> 💬 **Pat on the back if you got all four in**

Your Agent & Bedrock Experience

In one chat message, type:

1. **Your current AI/agent experience** (e.g., prompt scripts, prototypes, Bedrock APIs, or production agents)
2. **The layer you touch or care about most** (Runtime execution, Gateway/Tools, Identity & IAM, Memory, or Observability)
3. **One key production challenge or question** you want answered by this course

> 🌊 **Chat Waterfall**
>
> **Type your answer in the chat, but DO NOT hit enter yet.**
>
> *Wait for my countdown: 3... 2... 1... BLAST*

Your Agent & Bedrock Experience: What We Learned

If your message included these three things, we know what to emphasize:

- **Your starting point:** Bridging the gap from standalone prototypes to production systems
- **Your focus layer:** Runtime hosting, Gateway tool governance, Identity/IAM, Memory, or Observability
- **Your key questions:** We'll address these across the 5 core capabilities and in our final session recap

> 💬 **Pat on the back if you got all three in**

2. Introduction to AgentCore

What is Amazon Bedrock AgentCore?

A modular, managed platform for building, deploying, and operating agents securely at scale.

- **Framework-agnostic**: Works with LangGraph, CrewAI, LlamaIndex, etc.
- **Model portable**: Supports diverse foundation models.
- **Composable**: Services can be adopted independently.

Note: Framework portability reduces coupling but identity, memory, and telemetry remain platform-specific.

A Composable Agent Platform (Salesforce)

REAL-WORLD SCENARIO

Salesforce (Agentforce + Slack):

- Salesforce reports that Agentforce handled more than 1.5 million help-site requests and that its sales development representative agent worked more than 43,000 leads.
- Agentforce illustrates a composable business platform; AgentCore provides complementary infrastructure for AWS-hosted agents and tools rather than replacing Salesforce orchestration.

A Composable Agent Platform

SKIP | REAL-WORLD SCENARIO

Retail and e-commerce (AWS reference patterns):

- AWS demonstrates shopping agents that combine AgentCore Runtime, Strands Agents, and Amazon OpenSearch Service while keeping each capability independently replaceable.
- AWS's e-commerce MCP (Model Context Protocol) pattern exposes product search, ordering, reviews, and returns as governed tools instead of coupling every channel to custom integrations.

A Composable Agent Platform (Healthcare)

SKIP | REAL-WORLD SCENARIO

Healthcare (Stanford Health Care):

- Stanford embeds AI agents directly into Epic EHR workflows, generating personalized real-world evidence at the point of care without the physician asking a question.
- Cleveland Clinic runs Bayesian Health's AI platform across 13+ hospitals for real-time sepsis detection, analyzing labs, vitals, and clinical notes in the background.

3. The Five Focus Capabilities

1. AgentCore Runtime

A secure, serverless hosting environment for agent and tool code.

ISOLATION

Dedicated microVM (lightweight virtual machine) isolation per session.

WORKLOADS

Supports synchronous and asynchronous tasks.

BOUNDARY

Isolates compute, but your application backend must enforce user-to-session mapping.

2. AgentCore Gateway

A standardized, governed entry point for agents to invoke tools and models.

INTEGRATION

MCP (Model Context Protocol) aggregation for APIs and Lambda.

ROUTING

Direct HTTP and inference routing.

BOUNDARY

Agent-facing and tool-aware; does not replace Amazon API Gateway for client traffic.

3. AgentCore Identity

Manages inbound agent authentication and outbound service access.

PROVIDERS

Integrates with Amazon Cognito, Okta, Microsoft Entra ID.

ACCESS

Uses IAM SigV4 (Signature Version 4) for AWS service-to-service access.

BOUNDARY

Identity propagation isn't authorization alone; requires least-privilege IAM.

4. AgentCore Memory

Supplies state for context-aware behavior.

SHORT-TERM

Conversational events.

LONG-TERM

Information retained across sessions.

BOUNDARY

Needs tenant partitioning, retention policies, and data governance.

5. AgentCore Observability

Agent-specific tracing, metrics, and logs for debugging and operations.

TELEMETRY

CloudWatch-backed tracing for tokens, latency, and tool invocations.

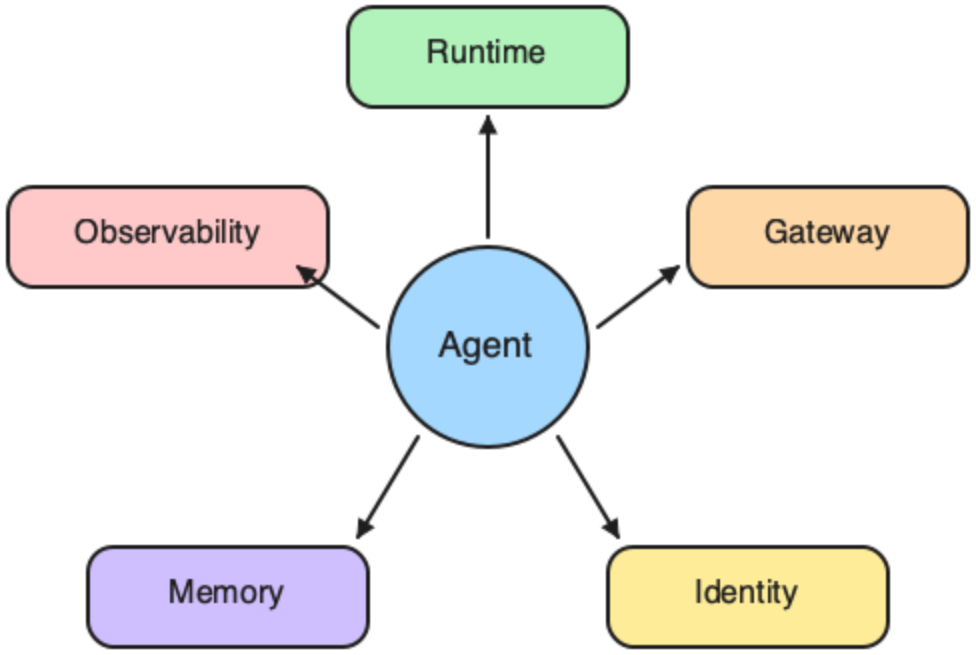
STANDARD

Emits OpenTelemetry-compatible telemetry.

BOUNDARY

Teams must define SLOs (Service Level Objectives), alarms, and handle sensitive payload data.

Complete AgentCore Taxonomy (1/2)



A composable set of services around the core agent

Runtime, Gateway, Identity, Memory, and Observability: five services wrapped around the agent. These are the **course focus**. AgentCore also includes

Complete AgentCore Taxonomy (2/2)

- **Harness**: Managed agent orchestration loop
- **Policy**: Centralized governance and compliance rules
- **Evaluations**: Behavioral testing and quality measurement
- **Registry**: Agent versioning and promotion (DRAFT → PUBLISHED)
- **Browser**: Secure web browsing for agents
- **Code Interpreter**: Sandboxed code execution
- **Payments**: Agent-mediated transactions
- **Optimization**: Cost and performance tuning

Course Focus

> **NOTE**
>
> This course covers Runtime, Gateway, Identity, Memory, and Observability in depth. The others are positioned for awareness.

Adopting AgentCore

AgentCore services are composable. You can adopt them independently. A team deploys an agent with just Runtime and Gateway. Six months later, they need identity propagation and memory.

A) They must redeploy everything: services must be adopted together
B) They can add Identity and Memory without touching their existing Runtime and Gateway setup
C) They should have adopted all five focus capabilities from day one
D) Only Runtime and Gateway are production-ready; the rest are experimental

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now.**

Adopting AgentCore: The Answer

B: You can add Identity and Memory without touching your existing Runtime and Gateway.

AgentCore is designed for incremental adoption. Each service is independently consumable: start with Runtime and Gateway, layer in Identity when you need attribution, add Memory when context matters. The five focus capabilities compose, they don't force a big-bang migration.

Adopting AgentCore

AgentCore services are composable — you can adopt them independently. A team deploys an agent with just Runtime and Gateway. Six months later, they need identity propagation and memory.

A) They must redeploy everything — services must be adopted together
B) They can add Identity and Memory without touching their existing Runtime and Gateway setup
C) They should have adopted all five focus capabilities from day one
D) Only Runtime and Gateway are production-ready; the rest are experimental

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now!**

Adopting AgentCore: The Answer

B — You can add Identity and Memory without touching your existing Runtime and Gateway.

AgentCore is designed for incremental adoption. Each service is independently consumable — start with Runtime and Gateway, layer in Identity when you need attribution, add Memory when context matters. The five focus capabilities compose, they don't force a big-bang migration.

Environment Readiness

- AWS Sandbox Environment
- Pre-configured IAM (Identity and Access Management) roles and permissions
- Required Tools: AWS CLI, Python

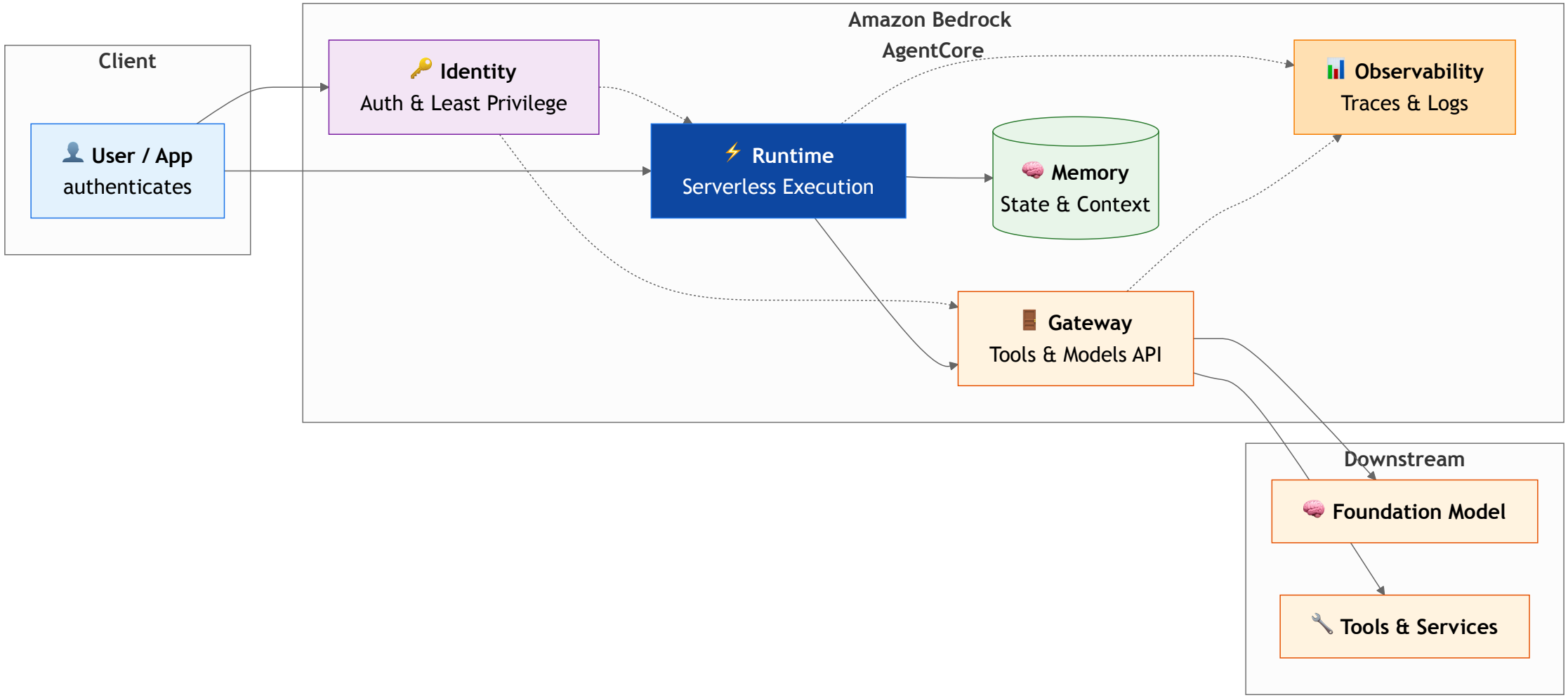
Check your access and verify the environment setup before we proceed.

Optional Breakout Lab

Transition to deck `01b-breakout-lab-first-request.md`

4. Enterprise Architecture

AgentCore Architecture



Shared Responsibility

Deploying an agent on managed infrastructure does not automatically make the agent safe or compliant.

- **Control Plane**: Immutable Runtime versions, canary releases.
- **Data Plane**: Memory partitioning, PII (Personally Identifiable Information) redaction.
- **Governance**: Security reviews, explicit approval for consequential actions.

Who Owns Agent Safety: Discussion

You deploy an agent on AgentCore Runtime with microVM (lightweight virtual machine) isolation, governed Gateway tool access, and CloudWatch tracing. An auditor asks: "If this agent makes a wrong decision, who is accountable?"

> **DISCUSS**
>
> Does managed infrastructure transfer responsibility for agent behavior to the cloud provider?

Who Owns Agent Safety: The Reality

The cloud provider secures the infrastructure. Your team owns the agent's behavior.

Layer	Provider Owns	You Own
Compute	MicroVM isolation	Code you deploy
Tool Access	Gateway allowlist enforcement	Which tools you allowlist
Identity	IAM policy evaluation	Policy contents, least privilege
Telemetry	Trace emission	SLOs, alerting, redaction

> Managed services give you better tools. They do not give you a safety warranty.

Infrastructure Alone Is Not Enough

REAL-WORLD SCENARIO

Salesforce (Customer Zero):

- Early Agentforce deployments showed that inconsistent CRM data can produce conflicting answers. Governance and data consolidation matter alongside model and prompt quality.
- Specialized agents with clear scope, owners, and escalation paths are easier to evaluate and operate than unrestricted generalists.

Infrastructure Alone Is Not Enough (Retail and E-commerce)

SKIP | REAL-WORLD SCENARIO

Retail and e-commerce (returns workflow):

- Runtime hosting does not decide whether a shopper owns an order or whether a refund is within policy. Those checks belong in deterministic application and service controls.
- Production readiness also requires idempotency, approval evidence, data retention, cost limits, and incident ownership.

Key Takeaways

- AgentCore is a composable set of services: Runtime, Gateway, Identity, Memory, and Observability.
- Harness handles orchestration; Runtime handles execution.
- Production readiness requires explicit least-privilege IAM, proper session mapping, and full observability.

> Reminder: **IAM** stands for Identity and Access Management.

Predict the Trace

The demo traces one request through Identity, Runtime, Gateway, Memory, and Observability. The agent calls a single tool: `get_order_status`.

How many tool calls will the end-to-end trace report?

A) 5: one per AgentCore capability traversed
B) 2: matching the two LLM (Large Language Model) calls in the trace
C) 1: the single `get_order_status` call
D) 0: Identity and Gateway don't count as tool calls

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now, with one reason.**

Demo: AgentCore Request Lifecycle

REFERENCE ONLY

Run `python3 day1/demos/demo-agentcore-lifecycle.py` (offline walkthrough).

- Identity: Caller authentication and context propagation
- Runtime: Session isolation and workload identity
- Gateway: Governed tool access via MCP
- Memory: Short-term and long-term context
- Observability: End-to-end trace with key signals

Predict the Trace: The Answer

C: 1 tool call (`get_order_status`).

The trace's `signals` block reports `tool_calls: 1` even though the request traverses five capabilities and makes two `gen_ai.client.operation` (LLM) calls. Capability boundaries are architectural: Identity, Runtime, Gateway, Memory, and Observability each govern part of the request, but they are not tool invocations. Only Gateway-routed calls to an actual tool count as tool calls. If the agent had called a second tool, `tool_calls` would read 2, independent of how many capabilities were involved.

End of Architecture Overview

Next up: Core Execution